# FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT
2

RULE – BASED AGENT

| | |
|---|---|
| Student Name | **Geofrey Miguel Erlano** |
| Section: | **AN41** |
| Professor: | **Ms. Crisola Tan** |
| Date: | **Oct 2, 2026** |

## I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

## II. TECHNICAL ASSESSMENT

### RULE – BASED AGENT

*Objective*

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.

3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This Technical Assessment (TA4) must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as TA2_lastname.pdf
   *(Example: TA2_Patilano.pdf)*
8. Submit the python file and PDF file.

Laboratory Task

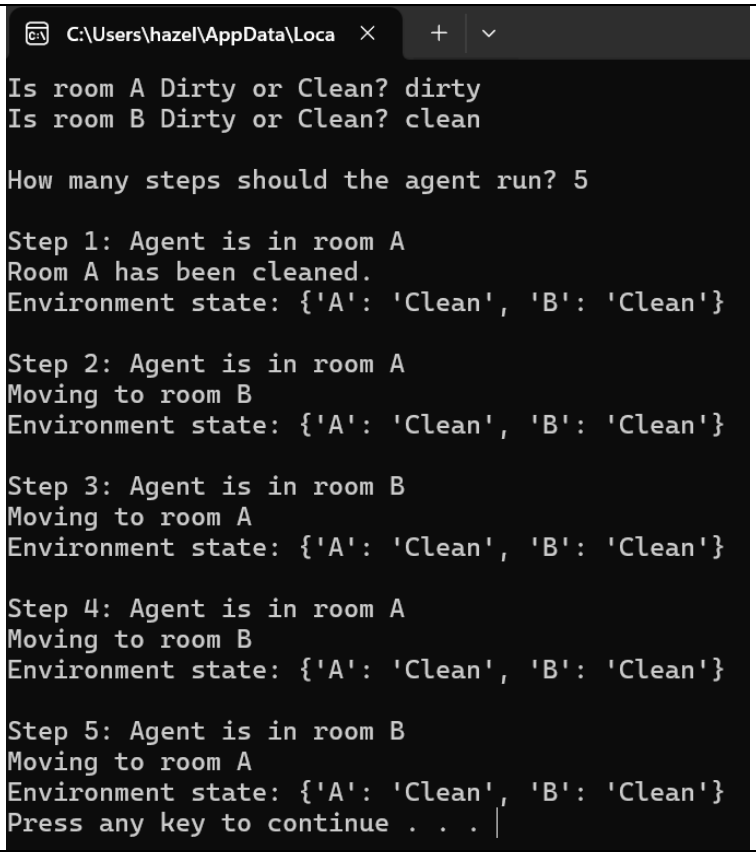
Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements
   - Loops (for loop)
   - Functions / methods
2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the initial state of each room (dirty or clean).
3. Write a function to check if a room is dirty and a function to clean a room.

Sample Output




Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the perceive-and-act method:
    - Rule 1: If the current room is dirty, clean it.
    - Rule 2: If the current room is clean, move to the other room.
3. Implement a move method to switch rooms.

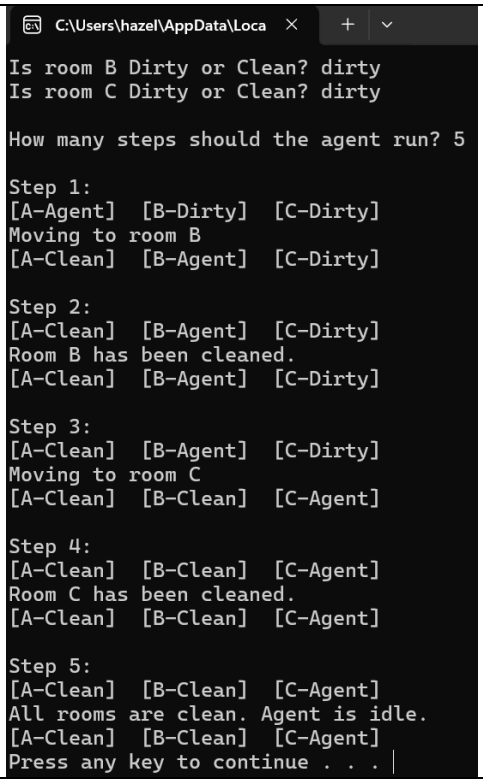
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a third room (C) and update the agent logic.
- Make the agent randomly choose which dirty room to clean next.
- Visualize the agent moving between rooms using a text-based grid.

Sample Output (Bonus Task)




Your Code - Laboratory Tasks

```
import random

class Environment:
    def __init__(self, state_a, state_b, state_c):
        # Task 1 & Bonus: Initialize rooms using a dictionary, now including Room C
        self.rooms = {'A': state_a, 'B': state_b, 'C': state_c}

    def is_dirty(self, room):
        # Task 1: Check if a room is dirty
        return self.rooms[room].lower() == 'dirty'

    def clean_room(self, room):
        # Task 1: Clean a room
        self.rooms[room] = 'Clean'

    def get_dirty_rooms(self):
        # Helper for Bonus Task to find all currently dirty rooms
        return [room for room, state in self.rooms.items() if state.lower() == 'dirty']
```

```
        # Align an arrow (^) exactly under the agent's current location
        if agent.location == room:
            padding = len(box) // 2
            agent_pointer += (" " * padding) + "^ Agent" + (" " * (padding - 7)) + "  "
        else:
            agent_pointer += " " * len(box)

    print("\n" + grid_display)
    print(agent_pointer)

def main():
    # Task 1: Allow user to set initial states
    print("Set the initial state of the rooms (enter 'Dirty' or 'Clean')")
    state_a = input("Room A: ").strip().capitalize()
    state_b = input("Room B: ").strip().capitalize()
    state_c = input("Room C: ").strip().capitalize() # Bonus room
```

```
class RuleBasedAgent:
    def __init__(self, starting_location='A'):
        # Task 2: Create a class for the agent
        self.location = starting_location

    def perceive_and_act(self, env):
        # Task 2: Implement the perceive-and-act method
        if env.is_dirty(self.location):
            # Rule 1: If current room is dirty, clean it
            env.clean_room(self.location)
            return "Cleaned"
        else:
            # Rule 2 / Bonus: If clean, move to another room (random dirty room if available)
            return self.move(env)

    def move(self, env):
        # Task 2 & Bonus: Implement a move method to switch rooms randomly
        dirty_rooms = env.get_dirty_rooms()

        if dirty_rooms:
            # Bonus: Randomly choose which dirty room to clean next
            self.location = random.choice(dirty_rooms)
            return "Moved (targeting a dirty room)"
        else:
            # If all rooms are clean, just randomly move to any other room
            available_rooms = [r for r in env.rooms.keys() if r != self.location]
            self.location = random.choice(available_rooms)
            return "Moved (all rooms are clean)"

def visualize_grid(env, agent):
    # Bonus: Visualize the agent moving between rooms using a text-based grid
    grid_display = ""
    agent_pointer = ""

    for room in ['A', 'B', 'C']:
        # Format the room block, e.g., [ A: Dirty ]
        state = env.rooms[room]
        box = f"[ {room}: {state:<5} ]  "
        grid_display += box
    env = Environment(state_a, state_b, state_c)
    agent = RuleBasedAgent(starting_location='A')

    # Task 3: Ask user how many steps to run
    try:
        steps = int(input("\nHow many steps should the simulation run? "))
    except ValueError:
        print("Please enter a valid number.")
        return

    # Task 3: Run the simulation in a loop
    for step in range(1, steps + 1):
        print(f"\n--- Step {step} ---")

        # Display the visual text grid before the action
        visualize_grid(env, agent)

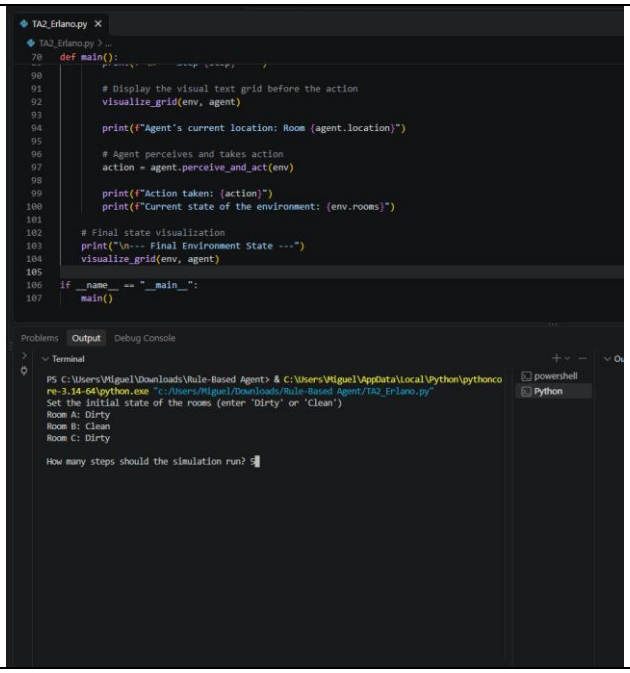
            print(f"Agent's current location: Room {agent.location}")

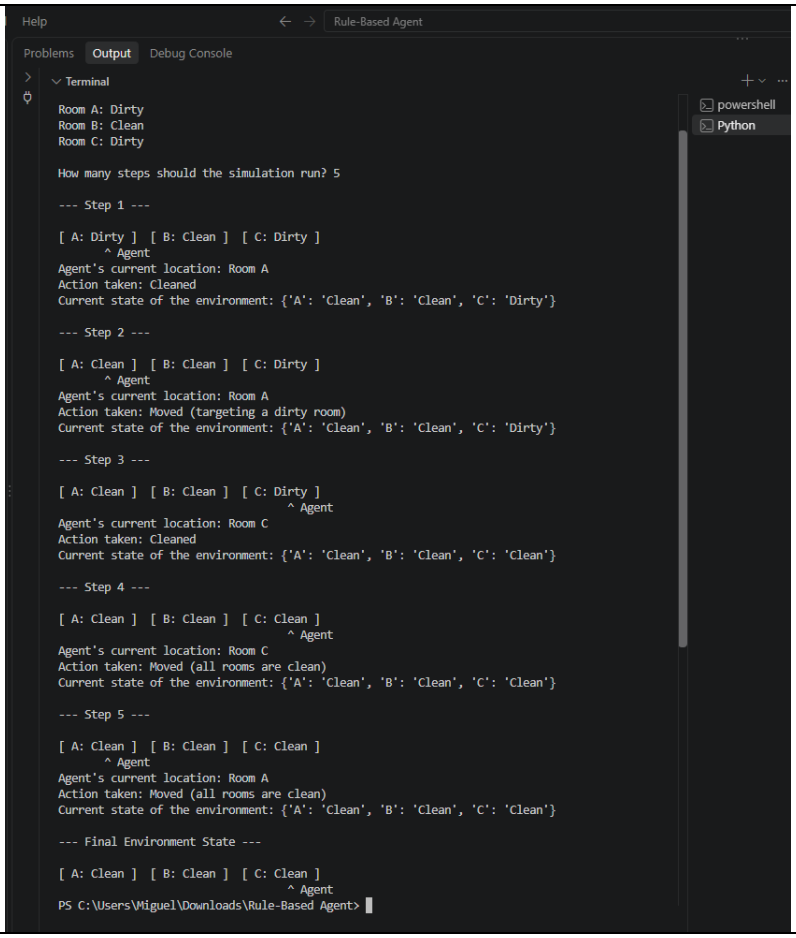
        # Agent perceives and takes action
        action = agent.perceive_and_act(env)

        print(f"Action taken: {action}")
            print(f"Current state of the environment: {env.rooms}")

    # Final state visualization
    print("\n--- Final Environment State ---")
    visualize_grid(env, agent)

if __name__ == "__main__":
    main()
```


TA2_Erlano.py
def main():
# Display the visual text grid before the action
visualize_grid(env, agent)
print(f"Agent's current location: Room {agent.location}")
# Agent perceives and takes action
action = agent.perceive_and_act(env)
print(f"Action taken: {action}")
print(f"Current state of the environment: {env.rooms}")
# Final state visualization
print("\n--- Final Environment State ---")
visualize_grid(env, agent)
if __name__ == "__main__":
main()
Problems
Output
Debug Console
Terminal
PS C:\Users\Miguel\Downloads\Rule-Based Agent> & C:\Users\Miguel\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/Miguel/Downloads/Rule-Based Agent/TA2_Erlano.py"
Set the initial state of the rooms (enter 'Dirty' or 'Clean')
Room A: Dirty
Room B: Clean
Room C: Dirty
How many steps should the simulation run? 5
powershell
Python


Help
Rule-Based Agent
Problems
Output
Debug Console
Terminal
Room A: Dirty
Room B: Clean
Room C: Dirty
How many steps should the simulation run? 5
--- Step 1 ---
[ A: Dirty ] [ B: Clean ] [ C: Dirty ]
^ Agent
Agent's current location: Room A
Action taken: Cleaned
Current state of the environment: {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}
--- Step 2 ---
[ A: Clean ] [ B: Clean ] [ C: Dirty ]
^ Agent
Agent's current location: Room A
Action taken: Moved (targeting a dirty room)
Current state of the environment: {'A': 'Clean', 'B': 'Clean', 'C': 'Dirty'}
--- Step 3 ---
[ A: Clean ] [ B: Clean ] [ C: Dirty ]
^ Agent
Agent's current location: Room C
Action taken: Cleaned
Current state of the environment: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
--- Step 4 ---
[ A: Clean ] [ B: Clean ] [ C: Clean ]
^ Agent
Agent's current location: Room C
Action taken: Moved (all rooms are clean)
Current state of the environment: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
--- Step 5 ---
[ A: Clean ] [ B: Clean ] [ C: Clean ]
^ Agent
Agent's current location: Room A
Action taken: Moved (all rooms are clean)
Current state of the environment: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
--- Final Environment State ---
[ A: Clean ] [ B: Clean ] [ C: Clean ]
^ Agent
PS C:\Users\Miguel\Downloads\Rule-Based Agent>
powershell
Python

**Task 4: Analysis Questions**

After running the simulation, answer the following:

1. What rules does the agent follow?
   **- The agent strictly follows two specific rules for its daily task. First, if its current room is dirty, it is required to clean it. Second, if the current room is already clean, the agent must move to the other room instead.**

2. How does the agent decide when to clean and when to move?
   **- The agent makes its decision by perceiving the current state of its environment. It uses a programmed rule to check the room's condition before taking any action. Therefore, seeing dirt triggers the cleaning command, while sensing a clean room triggers the movement command.**

3. What happens if both rooms are already clean at the start?
   **- If both rooms are completely clean at the very beginning, the agent will not clean anything. It will simply keep moving back and forth between the rooms endlessly. This behavior occurs because its only programmed instruction for a clean room is to move away.**

4. How would you modify the agent to handle more than two rooms?
   **- To handle more rooms, I would first update our environment dictionary to include new locations, such as a Room C. Furthermore, I would modify the agent's logic so that it can randomly choose which dirty room it should navigate to next.**